06 POINTERS AND REFERENCES • PRACTICE QUESTIONS

conceptual questions

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

Conceptual Questions — Module 06: Pointers and References (Questions 1–10)

Level 1 — Beginner

Q1. What is a pointer in C++? How does the value stored inside a pointer differ from the value stored inside an ordinary `int` variable?

Q2. Explain the exact roles of the address-of operator (`&`) and the dereference operator (`*`). Why does `&42` fail to compile?

Q3. Why should modern C++ programs use `nullptr` instead of `NULL` or `0` to initialize null pointers?

Level 2 — Intermediate

Q4. On a 64-bit system, `sizeof(char*)` , `sizeof(int*)` , and `sizeof(double*)` are all `8` bytes. Since all pointers have the same byte size, why does C++ require pointers to have distinct types (`char*` vs. `int*` vs. `double*`) instead of making every pointer a generic address?

Q5. Differentiate between:

1. `const int* p`
2. `int* const p`
3. `const int* const p`

State which operations (`p = &other;` and `*p = 100;`) are legal or illegal for each.

Q6. What is a `void*` (generic pointer)? List two operations that are legal on `int*` but are compile-time errors on `void*` .

Level 3 — Advanced

Q7. Distinguish clearly between a **wild pointer**, a **null pointer**, a **dangling pointer**, and a **memory leak**. Which of these can be checked safely with `if (p ≠ nullptr)` and which cannot?

Q8. State at least six technical differences between a **pointer** (`int*`) and a **reference** (`int&`). Why is an array of references (`int& arr[5]`) forbidden by the C++ standard?

Level 4 — University Examination Level

Q9. Explain the difference between **operator precedence**, **operator associativity**, and **order of evaluation (sequencing)** in C++. Why is operator precedence alone insufficient to explain why `*p++ + *p++` causes Undefined Behavior?

Q10. Compare raw owning pointers (`new` / `delete`) with modern C++ smart pointers (`std::unique_ptr` , `std::shared_ptr` , and `std::weak_ptr`). How does `std::weak_ptr` solve the circular reference problem between two `std::shared_ptr` objects?